CHAPTER 1

REVIEW OF C++ PROGRAMMING

CHAPTER ESSENCE AT A GLANCE:
C++ combines low-level machine efficiency with high-level procedural abstraction through strongly typed variables, expressive tokens, deterministic and indeterminate control loops, and multi-way branch selection.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Token Classification and Lexical Units:** Every C++ program is parsed into elementary tokens comprising keywords reserved by the language, user-defined identifiers obeying strict naming rules, typed literals representing constants, operators defining computations, and punctuators structuring syntax.

• **Type Promotion and Implicit Coercion:** Expressions combining operands of mixed data types evaluate according to strict promotion hierarchies, where lower-rank integer types automatically widen to higher-precision floating-point types before arithmetic evaluation to prevent precision truncation.

• **Selection Branching Mechanics:** Conditional decision making directs program control through two-way if-else branches or multi-way switch-case dispatch tables evaluated on integral or character selectors with explicit break execution.

• **Iteration Control and Loop Invariants:** Repetitive execution utilizes entry-controlled for and while loops that test conditions prior to body entry, and exit-controlled do-while loops guaranteeing at least one execution pass before evaluating termination predicates.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Ternary Selection Equivalence} = (\text{Expression}_1 \ ? \ \text{Expression}_2 : \text{Expression}_3) \equiv \textbf{if}\ (E_1)\ \{E_2;\}\ \textbf{else}\ \{E_3;\}$$

Conditional Operator Equivalence

$$\text{Implicit Type Promotion Hierarchy} = \frac{\text{bool} \to \text{char} \to \text{int} \to \text{unsigned int} \to \text{long}}{\text{unsigned long} \to \text{float} \to \text{double} \to \text{long double}}$$

Standard Data Type Coercion Order

$$\text{Loop Iteration Count} = \frac{\text{Upper Limit} - \text{Lower Limit}}{\text{Step Size}} + 1 \quad \text{(For Inclusive Bounded Loops)}$$

Deterministic Loop Execution Formula

$$\text{Switch Dispatch Architecture} = \textbf{switch}(\text{Integer Expr}) \longrightarrow \textbf{case}\ \text{Constant}_k : \text{Statements};\ \textbf{break};$$

Multi-Way Jump Table Mechanics

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Never place a semicolon immediately after the conditional expression of an if statement or loop header; doing so creates an empty statement and severs the intended code block.

• **Alert:** Remember that switch expression and case labels must evaluate strictly to integer or character constants; floating-point numbers and string literals are syntactically invalid.

• **Alert:** Always include a break statement at the conclusion of every switch case unless intentional fall-through execution across multiple cases is explicitly intended.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Distinguish between keywords and identifiers in C++ with two examples of each. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Keywords: Reserved words with pre-defined meanings in the C++ compiler that cannot be altered or used as variable names (e.g. int, return, switch). [1 Mark]

• Identifiers: User-defined names given to program elements like variables, functions, and arrays, which must begin with a letter or underscore (e.g. rollNumber, totalMarks). [1 Mark]

2. Compare entry-controlled loops and exit-controlled loops in C++ regarding their condition checking and minimum execution count. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Entry-Controlled Loops: The test expression is evaluated before entering the loop body (e.g. for and while loops). If the condition evaluates to false initially, the loop body executes zero times. [1 Mark]

• Exit-Controlled Loops: The test expression is evaluated at the end of the loop body (e.g. do-while loop). The body executes at least once even if the condition is false initially. [1 Mark]

3. Trace the execution of the following C++ code snippet and write its exact output: int a = 1, s = 0; while (a <= 5) { s = s + a; a = a + 2; } cout << s << ' ' << a; [Score: 3 Marks]

Answer & Key Valuation Steps:

• Iteration 1: a is 1 (condition true), s becomes 0 + 1 = 1, a increments to 3. [1 Mark]

• Iteration 2: a is 3 (condition true), s becomes 1 + 3 = 4, a increments to 5. [1 Mark]

• Iteration 3 and Exit: a is 5 (condition true), s becomes 4 + 5 = 9, a increments to 7. When a is 7, condition evaluates to false. Output displayed is: 9 7. [1 Mark]

4. Explain why the break statement is essential inside switch-case blocks and describe what happens when break is omitted. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Purpose of break: The break statement forces immediate termination of the switch construct and transfers control to the statement following the switch closing brace. [1 Mark]

• Fall-Through Behavior: When break is omitted, execution falls through into subsequent case blocks and executes their statements unconditionally until a break is encountered or the switch ends. [1 Mark]

• SCERT Scoring Tip: Illustrate with a concise case block showing fall-through to secure full valuation points. [1 Mark]

5. Identify and correct three syntax errors in the following C++ code fragment: void main() { int 1num, val = 10; if (val = 10); cout << val; } [Score: 3 Marks]

Answer & Key Valuation Steps:

• Error 1 (Invalid Identifier): Variable name '1num' starts with a digit. Correction: Rename to 'num1' or similar valid identifier. [1 Mark]

• Error 2 (Assignment in Condition): 'val = 10' is an assignment operator instead of equality comparison. Correction: Replace with relational equality 'val == 10'. [1 Mark]

• Error 3 (Semicolon after If): The semicolon after if condition creates an empty statement. Correction: Remove the semicolon following '(val == 10)'. [1 Mark]

6. Write a complete C++ program to input a positive integer and determine whether it is a prime number. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Header and Variable Setup: Include iostream, declare namespace std, define main(), and initialize int num, i, isPrime = 1. [1 Mark]

• Loop Logic: Input num; if num <= 1 set isPrime = 0; else loop for (i = 2; i * i <= num; i++) if (num % i == 0) { isPrime = 0; break; }. [1 Mark]

• Result Display: If isPrime is 1, print 'Prime Number'; else print 'Composite Number'. Return 0 to terminate successfully. [1 Mark]

7. Discuss jump statements in C++: break, continue, goto, and return, highlighting their specific execution control behavior. **[Score: 4 Marks]**

Answer & Key Valuation Steps:

• break Statement: Terminates the innermost enclosing loop or switch construct immediately and transfers control to the next sequential statement outside the block. **[1 Mark]**

• continue Statement: Skips the remaining statements within the current loop iteration and transfers control directly to the loop condition evaluation or update expression. **[1 Mark]**

• goto Statement: Causes an unconditional jump of control to a specified named label within the same function scope, though discouraged in structured programming. **[1 Mark]**

• return Statement: Terminates execution of the current function immediately and returns program control along with an optional value back to the calling function. **[1 Mark]**